A Systematic Benchmark of Persistent Homology Pipelines for Classification

Zachariah Attila Markusson

August 2026

Independent Research

Abstract

Persistent homology classification pipelines have three stages — filtration, vectorization, and classifier — each with multiple available methods. Which stage matters most? The literature's most widely cited answer, from Conti et al. [8], is that filtration choice dominates: switching filtrations on MNIST images swung accuracy from 18% to 94%. We show that this finding does not generalise. On ECG200 time series with Takens embedding, vectorization choice dominates classification accuracy with a marginal range of 6.1 percentage points (across 7 vectorizers, 95% CI [4.8, 7.4]), while filtration choice contributes only 0.7pp (across 4 filtrations, 95% CI [0.2, 1.1]). Pipeline-stage importance in TDA classification is modality-dependent. The sweep comprised 4 filtrations, 7 vectorizers, and 4 classifiers (the framework is extensible to 7 filtrations, 11 vectorizers, and 4 classifiers via YAML-only changes).

We demonstrate this on one synthetic (sphere/torus at four noise levels) and two real-world datasets (ECG200, MNIST binary), with 616 completed configurations (of 672 possible; 56 excluded where the point-cloud filtrations Weak Alpha and Sparse Rips are incompatible with image data). The topological signal on sphere/torus point clouds survives Gaussian noise up to $\sigma = 0.30$ at 99.85% mean accuracy across the 112 configurations at that level: classification accuracy survives noise levels where the stability bound's worst-case guarantee would permit degradation, and the $\beta_1 = 0$ vs. $\beta_1 = 2$ distinction remains classifiable. On MNIST, cubical filtration outperforms Vietoris-Rips by 1.8 percentage points (98.0% vs. 96.25%), confirming that filtration choice matters for image data. The Alpha complex is 11–21% faster than Vietoris-Rips at identical accuracy on 100-point 3D clouds under the giotto-tda implementation. Simple persistence statistics match kernel-based vectorizers on real data despite requiring zero hyperparameters. All code, configuration, and result schemas are provided; 616 completed configurations across 6 dataset instances, 4 filtrations, 7 vectorizers, and 4 classifiers.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Motivation

1.1 Topological Data Analysis and the Pipeline Problem

Topological Data Analysis (TDA) extracts shape invariants from data: connected components, cycles, and voids that persist across multiple scales. Unlike traditional machine learning which operates on raw coordinates or pixel intensities, TDA captures the underlying "shape" of the data — information invariant under stretching and bending. Persistent homology, the workhorse of TDA, tracks the birth and death of topological features as a scale parameter varies.

Practitioners face a combinatorial problem: the TDA classification pipeline has three stages — filtration, vectorization, and classification — each with 5–15 available methods (surveys: Hatwar and Thangaraj [10]). The literature's most widely cited answer to "which stage matters most?" comes from Conti et al. [8], who showed that filtration choice on MNIST images can swing accuracy from 18% to 94% — a finding that has been interpreted as a general principle: "filtration dominates."

This paper argues that principle does not generalise. Pipeline-stage importance is modality-dependent. We demonstrate this using a modular benchmarking framework that varies all three pipeline stages simultaneously in a controlled factorial sweep: 6 dataset instances $\times$ 4 filtrations (Vietoris-Rips, Alpha, Sparse Rips, Cubical) $\times$ 7 vectorizers $\times$ 4 classifiers — 672 possible, 616 completed (56 excluded where point-cloud filtrations are incompatible with image data). The framework is extensible to 7 filtrations, 11 vectorizers, and 4 classifiers via YAML-only configuration changes; standardised harnesses of this kind are the direction the community has moved towards [18].

1.2 The Persistent Homology Pipeline

Whether persistent homology features help classification at all has itself been questioned [19]; this benchmark assumes the features are informative and asks which pipeline stage extracts them most effectively.

A persistent homology classification pipeline transforms raw data through three stages:

1. **Filtration.** Raw data is converted into a nested sequence of simplicial complexes parameterised by a scale ε. Topological features appear (birth at b) and disappear (death at d). Output: a *persistence diagram*, a multiset of (b, d) pairs.

2. **Vectorization.** Diagrams are multisets of variable cardinality and cannot be fed directly into classifiers. Vectorization methods map each diagram to a fixed-length feature vector.

3. **Classification.** The vectorized features are passed to a standard classifier which learns a decision boundary.

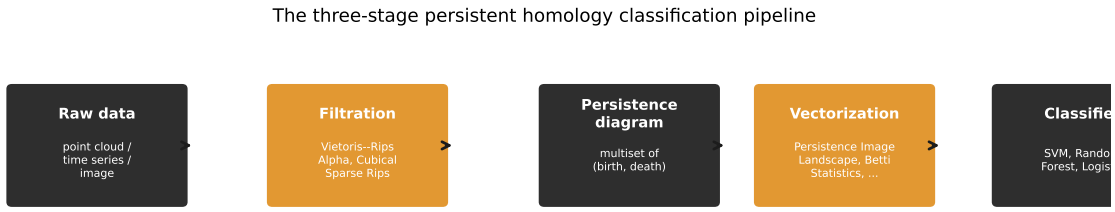

Figure 1.1: The three-stage persistent homology classification pipeline.

1.3 Contributions

This paper demonstrates that pipeline-stage importance in TDA classification is modality-dependent, using a modular benchmarking framework built for that purpose. Specifically:

1. **Finding.** On ECG200 time series with Takens embedding, vectorization dominates classification accuracy (6.1pp marginal range across 7 vectorizers, 95% CI [4.8, 7.4]) while filtration is statistically neutral (0.7pp across 4 filtrations, 95% CI [0.2, 1.1]). This qualifies the widely cited claim that filtration always dominates — that finding holds for image data but does not generalise. Pipeline-stage importance is fundamentally modality-dependent.

2. **Framework.** The finding was produced by a benchmarking framework that varies all three pipeline stages simultaneously: 4 filtrations, 7 vectorizers, and 4 classifiers executed in a 616-configuration factorial sweep (of 672 possible; 56 exclusions where point-cloud filtrations are incompatible with image data), with YAML-driven configuration and normalised SQLite storage. The framework is extensible to 7 filtrations, 11 vectorizers, and 4 classifiers via YAML-only changes. Full configuration and schema in Appendices A–B; code implementations in Appendix C.

Chapter 2 develops the mathematical machinery needed to understand why these pipeline stages interact as they do.

Chapter 2

Mathematical Foundations of Persistent Topology

This chapter develops the algebraic and geometric machinery underlying persistent homology. We progress from simplicial complexes through chain groups, homology vector spaces, filtrations, and persistence modules, concluding with the stability theorem and vectorization methods that convert diagrams into classifier-ready feature vectors.

2.1 Simplicial Complexes

Definition 2.1 (Simplex)**.** A *k-simplex* $\sigma = [v_0, v_1, \ldots, v_k]$ is the convex hull of $k+1$ affinely independent points in $\mathbb{R}^d$ ($d \geq k$). A 0-simplex is a vertex, a 1-simplex an edge, a 2-simplex a triangle, a 3-simplex a tetrahedron. A *face* of σ is the convex hull of any non-empty subset of its vertices.

Definition 2.2 (Simplicial Complex)**.** A *simplicial complex* K is a finite collection of simplices such that:

1. If $\sigma \in K$ and τ is a face of σ, then $\tau \in K$ (closure under taking faces).
2. If $\sigma, \tau \in K$, then $\sigma \cap \tau$ is either empty or a face of both.

The *k-skeleton* $K^{(k)}$ is the subcomplex of all simplices of dimension at most k. The 1-skeleton is a graph in the usual sense.

2.2 Filtrations and Complex Constructions

Definition 2.3 (Filtration)**.** A *filtration* of a simplicial complex K is a nested sequence of subcomplexes indexed by a real parameter $\varepsilon \geq 0$:

$$\emptyset = K_{\varepsilon_0} \subseteq K_{\varepsilon_1} \subseteq \cdots \subseteq K_{\varepsilon_m} = K, \tag{2.1}$$

where $\varepsilon_0 < \varepsilon_1 < \cdots < \varepsilon_m$.

Three filtrations are used in the benchmark:

Vietoris-Rips complex $\mathrm{VR}_\varepsilon(X)$. For a finite point cloud $X \subset \mathbb{R}^d$, a k-simplex $[x_0, \ldots, x_k]$ is included if $\|x_i - x_j\| \leq \varepsilon$ for all $0 \leq i < j \leq k$. The Rips complex is simple to compute but large: for n points, the number of k-simplices scales as $O(n^{k+1})$. Specifically, the number of 2-simplices (triangles) in the full Rips complex is $O(n^3)$, which drives the runtime differences observed in Chapter 4.

Alpha complex $A_\varepsilon(X)$. Constructed via the intersection of Voronoi cells with metric balls. For each point $x_i \in X$, let $V_i = \{y \in \mathbb{R}^d : \|y - x_i\| \leq \|y - x_j\| \text{ for all } j \neq i\}$ be its closed Voronoi cell, and define $R_i(\varepsilon) = V_i \cap \overline{B}_\varepsilon(x_i)$ where $\overline{B}_\varepsilon(x_i)$ is the closed ball of radius $\varepsilon/2$. The Alpha complex is the nerve of the cover $\{R_i(\varepsilon)\}_{i=1}^n$. The Nerve Theorem (see below) guarantees homotopy equivalence to the union of balls.

Theorem 2.1 (Nerve Theorem — Leray 1945 [12]). *Let $\mathcal{U} = \{U_i\}_{i \in I}$ be a finite open cover of a topological space M such that every non-empty intersection $\bigcap_{j \in J} U_j$ is contractible. Then the nerve $N(\mathcal{U})$ is homotopy equivalent to $\bigcup_{i \in I} U_i$.*

For the Alpha cover $\{R_i(\varepsilon)\}$, the sets $R_i(\varepsilon)$ are convex polyhedra in $\mathbb{R}^d$, so all finite intersections are convex and thus contractible. The Nerve Theorem therefore guarantees that $A_\varepsilon(X) \simeq \bigcup_i \overline{B}_{\varepsilon/2}(x_i)$. In $\mathbb{R}^3$, the Alpha complex is a subcomplex of the Delaunay triangulation with $O(n^2)$ simplices, compared to $O(n^3)$ for the full Rips complex.

Sparse Rips complex. A subsampled approximation of VR that retains only a sparse subset of edges and higher simplices, designed for point clouds with $n \geq 10^3$ points where full VR computation is prohibitive. The sparsification parameter ϵ controls the approximation fidelity.

2.3 Homology Groups and Persistent Homology

Chain Complexes and Homology

To define persistent homology formally, we first construct the algebraic machinery of simplicial homology over the field $\mathbb{Z}_2 = \mathbb{Z}/2\mathbb{Z}$.

Definition 2.4 (Chain Group). For a simplicial complex K, the *k-th chain group* $C_k(K; \mathbb{Z}_2)$ is the $\mathbb{Z}_2$-vector space whose basis elements are the k-simplices of K. An element $c \in C_k(K; \mathbb{Z}_2)$ is a formal sum $c = \sum_{\sigma \in K^{(k)}} a_\sigma \sigma$ with coefficients $a_\sigma \in \mathbb{Z}_2$.

Definition 2.5 (Boundary Operator). The *k-th boundary operator* $\partial_k : C_k(K; \mathbb{Z}_2) \to$

$C_{k-1}(K;\mathbb{Z}_2)$ is the linear map defined on a k-simplex $\sigma = [v_0, \ldots, v_k]$ by:

$$\partial_k[v_0, \ldots, v_k] = \sum_{i=0}^{k} [v_0, \ldots, \hat{v}_i, \ldots, v_k], \tag{2.2}$$

where $\hat{v}_i$ denotes omission of v_i, and the sum is taken over $\mathbb{Z}_2$ (so $1+1=0$). For $k=0$, $\partial_0 = 0$ by convention.

The fundamental property of the boundary operator is $\partial_k \circ \partial_{k+1} = 0$ for all $k \geq 0$. Intuitively, the boundary of a boundary is empty: the edges bounding a triangle form a closed loop with no endpoints.

Definition 2.6 (Homology Group)**.** For a simplicial complex K, define:

- $Z_k(K) = \ker \partial_k$ — the *k-cycles* (chains with zero boundary).
- $B_k(K) = \operatorname{im} \partial_{k+1}$ — the *k-boundaries* (chains that are boundaries of $(k+1)$-chains).

Since $\partial_k \circ \partial_{k+1} = 0$, we have $B_k(K) \subseteq Z_k(K)$. The *k-th homology group* is the quotient vector space:

$$H_k(K) = \frac{Z_k(K)}{B_k(K)} = \frac{\ker \partial_k}{\operatorname{im} \partial_{k+1}}. \tag{2.3}$$

Elements of $H_k(K)$ are equivalence classes of k-cycles, where two cycles are equivalent if they differ by a boundary. The dimension of $H_k(K)$ as a $\mathbb{Z}_2$-vector space is the *k-th Betti number*: $\beta_k(K) = \dim_{\mathbb{Z}_2} H_k(K)$. Informally, β_0 counts connected components, β_1 counts 1-dimensional holes (cycles), β_2 counts 2-dimensional voids.

Persistent Homology

A filtration $\{K_\varepsilon\}_{\varepsilon \geq 0}$ induces a sequence of inclusion maps $i^{\varepsilon,\varepsilon'} : K_\varepsilon \hookrightarrow K_{\varepsilon'}$ for $\varepsilon \leq \varepsilon'$, which in turn induce linear maps on homology:

$$f_k^{\varepsilon,\varepsilon'} : H_k(K_\varepsilon) \to H_k(K_{\varepsilon'}). \tag{2.4}$$

Definition 2.7 (Persistent Homology Group)**.** The *k-th persistent homology group* from ε to ε' is:

$$H_k^{\varepsilon,\varepsilon'}(K) = \operatorname{im} f_k^{\varepsilon,\varepsilon'}. \tag{2.5}$$

A homology class $\gamma \in H_k(K_{\mathrm{b}})$ is *born* at $\varepsilon = \mathrm{b}$ if $\gamma \notin \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{b}}$ for any $\delta > 0$, and *dies* at $\varepsilon = \mathrm{d} > \mathrm{b}$ if $f_k^{\mathrm{b},\mathrm{d}}(\gamma) \in \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{d}}$ for some $\delta > 0$ but $f_k^{\mathrm{b},\mathrm{d}-\delta}(\gamma) \notin \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{d}-\delta}$. If γ never dies, we set $\mathrm{d} = \infty$.

Definition 2.8 (Persistence Diagram)**.** The *persistence diagram* $\mathrm{Dgm}_k(K)$ is the multiset of points $(\mathrm{b}, \mathrm{d}) \in \mathbb{R}^2 \cup \{\infty\}$ where $\mathrm{b} < \mathrm{d}$, together with infinitely many copies of each point on the diagonal $\Delta = \{(x, x) : x \geq 0\}$. The *persistence* of a feature is $\mathrm{p} = \mathrm{d} - \mathrm{b}$.

Example. A sphere S^2 has $\beta = (1, 0, 1)$: one component, no 1-cycles, one 2-void. A torus T^2 has $\beta = (1, 2, 1)$: one component, two independent 1-cycles (the meridional and longitudinal loops), one 2-void. This $\beta_1 = 0$ versus $\beta_1 = 2$ difference is the topological signal driving the sphere-versus-torus classification in Chapter 4.

2.4 Stability of Persistence Diagrams

The stability theorem guarantees that small perturbations of the input data produce proportionally small changes in the persistence diagram — essential for any pipeline handling noisy data.

Definition 2.9 (Bottleneck Distance)**.** The *bottleneck distance* between two persistence diagrams D_1 and D_2 is:

$$d_B(D_1, D_2) = \inf_{\gamma : D_1 \to D_2} \sup_{p \in D_1} \|p - \gamma(p)\|_\infty, \tag{2.6}$$

where γ ranges over all bijections from D_1 to D_2 (including pairings with the diagonal Δ), and $\|\cdot\|_\infty$ is the L^∞ norm on $\mathbb{R}^2$.

Definition 2.10 (Gromov-Hausdorff Distance)**.** For compact metric spaces X and Y, the *Gromov-Hausdorff distance* is:

$$d_{GH}(X, Y) = \frac{1}{2} \inf_{Z, f, g} d_H^Z(f(X), g(Y)), \tag{2.7}$$

where the infimum is taken over all metric spaces Z and all isometric embeddings $f : X \hookrightarrow Z$, $g : Y \hookrightarrow Z$, and d_H^Z is the Hausdorff distance in Z: $d_H^Z(A, B) = \max\{\sup_{a \in A} \inf_{b \in B} d_Z(a, b),\ \sup_{b \in B} \inf_{a \in A} d_Z(a, b)\}$.

Theorem 2.2 (Stability Theorem — Cohen-Steiner et al. 2007 [7])**.** *Let X, Y be totally bounded metric spaces. Then:*

$$d_B(\mathrm{Dgm}_k(X), \mathrm{Dgm}_k(Y)) \leq 2\, d_{GH}(X, Y) \tag{2.8}$$

for all $k \geq 0$, where $\mathrm{Dgm}_k(\cdot)$ denotes the k-dimensional persistence diagram of the Vietoris-Rips or Čech filtration.

This theorem is applied in §4.2: for n points with additive Gaussian noise $\eta_i \sim \mathcal{N}(0, \sigma^2 I_3)$, the expected maximum perturbation is $\mathbb{E}[\max_i \|\eta_i\|] \approx \sigma\sqrt{2\log n}$ (via extreme value theory). With high probability the Gromov-Hausdorff distance is bounded by $O(\sigma\sqrt{\log n})$, so the bottleneck distance between clean and noisy diagrams is at most $O(\sigma\sqrt{\log n})$. For $n = 100$ (points per cloud after subsampling) and $\sigma = 0.15$, this gives $d_B \lesssim 0.46$; at $\sigma = 0.30$, $d_B \lesssim 0.91$. We compare this bound against the observed persistence ranges of the torus's β_1 features.

2.5 Vectorization Methods

Vectorization converts a persistence diagram into a fixed-length feature vector for classification. We use the taxonomy of Ali et al. [2]; comparative studies include Barnes et al. [3], Perea et al. [13], and Sulowska [15].

Persistence diagrams are multisets of variable cardinality. To use them with standard classifiers, we must map each diagram to a fixed-length vector in a normed space. We present four methods, ordered by increasing complexity.

Persistence Statistics. For each homology dimension k, compute the mean, standard deviation, minimum, and maximum of births, deaths, and lifespans ($\mathrm{p} = \mathrm{d} - \mathrm{b}$) across all points in Dgm_k. This yields $4 \times 3 = 12$ features per dimension. Despite zero hyperparameters, this method is competitive with kernel-based approaches [2].

Betti Curves [6]. The Betti curve $\beta_k(\varepsilon)$ counts the number of k-dimensional features with $\mathrm{b} \le \varepsilon < \mathrm{d}$. Evaluated at n_{bins} equally spaced points in $[\varepsilon_{\min}, \varepsilon_{\max}]$, this produces n_{bins} features per homology dimension.

Persistence Landscapes [4]. For each point $(\mathrm{b}, \mathrm{d}) \in \mathrm{Dgm}_k$, define the tent function:

$$\Lambda_{(\mathrm{b},\mathrm{d})}(t) = \max\big(0,\, \min(t - \mathrm{b},\, \mathrm{d} - t)\big). \tag{2.9}$$

The *m-th persistence landscape* is the function $\lambda_m : \mathbb{R} \to \mathbb{R}$ given by:

$$\lambda_m(t) = m\text{-}\max_{p \in \mathrm{Dgm}_k} \Lambda_p(t), \tag{2.10}$$

where m-max denotes the m-th largest value. Each λ_m is 1-Lipschitz. The sequence $(\lambda_m)_{m \in \mathbb{N}}$ resides in the Banach space $L^p(\mathbb{N} \times \mathbb{R})$ for any $1 \le p \le \infty$, enabling classical statistical operations (means, variances, confidence intervals) directly on the vectorized representation. With n_{layers} layers evaluated at n_{bins} points, this produces $n_{\text{layers}} \times n_{\text{bins}}$ features.

Persistence Images [1]. Transform each point $(\mathrm{b}, \mathrm{d}) \in \mathrm{Dgm}_k$ to birth-persistence coordinates (b, p) where $\mathrm{p} = \mathrm{d} - \mathrm{b}$. Define a weighting function $w : \mathbb{R} \times \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$, typically:

$$w(\mathrm{b}, \mathrm{p}) = \begin{cases} \mathrm{p}/\mathrm{p}_{\max}, & \mathrm{p} \leq \mathrm{p}_{\max}, \\ 1, & \text{otherwise}, \end{cases} \tag{2.11}$$

where $\mathrm{p}_{\max}$ is a fixed constant (set to the 95th percentile of persistences in our implementation). The persistence surface is:

$$\rho(x, y) = \sum_{(\mathrm{b},\mathrm{p}) \in T(\mathrm{Dgm}_k)} w(\mathrm{b}, \mathrm{p})\, \phi_{(\mathrm{b},\mathrm{p})}(x, y), \tag{2.12}$$

where $\phi_{(\mathrm{b},\mathrm{p})}(x, y) = \frac{1}{2\pi\sigma^2} \exp\!\left(-\frac{(x-\mathrm{b})^2+(y-\mathrm{p})^2}{2\sigma^2}\right)$ is a 2D Gaussian kernel. Evaluating ρ on an $n_{\text{bins}} \times n_{\text{bins}}$ grid yields n_{bins}^2 features per homology dimension.

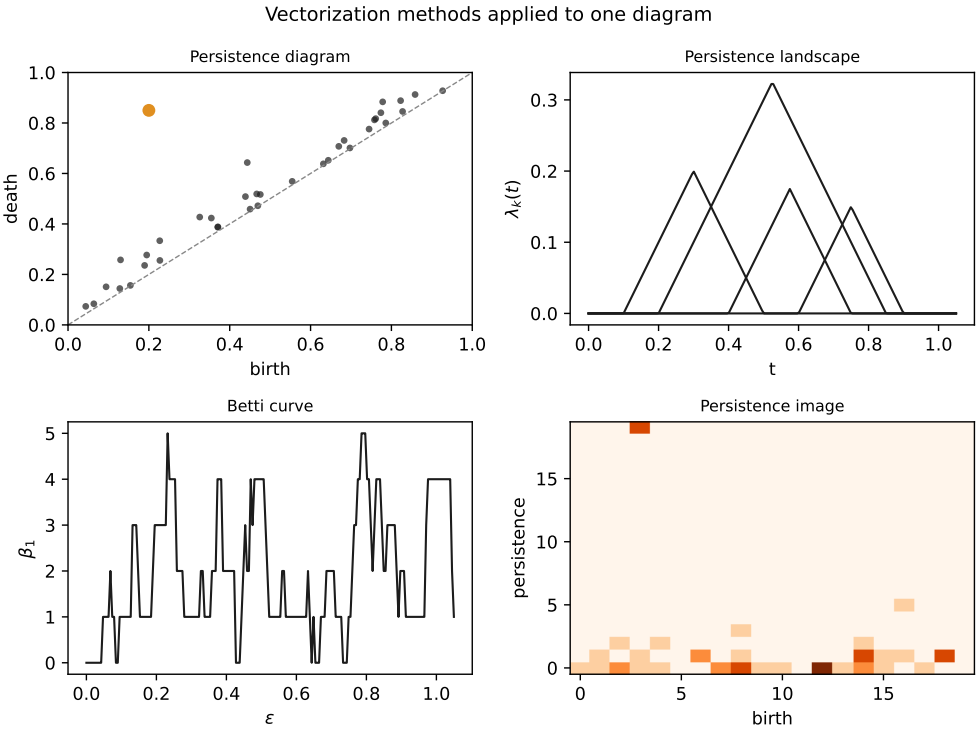

Figure 2.1: The four vectorization methods applied to one diagram.

2.6 Mapping Theory to Benchmark Variables

The preceding definitions directly parameterise the benchmark executed in Chapter 4:

- **Simplicial complex size** (§2.1–§2.2): VR's $O(n^3)$ 2-simplex count versus Alpha's $O(n^2)$ drives the 11–21% wall-clock speed gap measured in §4.3. The Nerve Theorem guarantees that Alpha's speed advantage does not cost geometric fidelity.

- **Filtration choice** (§2.2): VR, Alpha, Sparse Rips, and Cubical constitute the four filtrations varied in the factorial sweep of §4.1.
- **Persistence and stability** (§2.3–§2.4): The probabilistic bound $d_B \lesssim \sigma\sqrt{2\log n}$ predicts that $\sigma = 0.15$ noise perturbs diagrams by $\lesssim 0.46$, while torus β_1 features have persistence 0.5–0.8. §4.2 confirms that classification survives this perturbation.
- **Vectorization** (§2.5): Persistence Statistics, Betti Curves, Landscapes, and Images constitute the four vectorizers varied in §4.1, spanning the complexity spectrum from zero-parameter to kernel-based.
- **Classifiers** (§3.1): Logistic Regression (linear baseline), SVM-RBF, and Random Forest constitute the three classifiers in the sweep. §4.1 tests whether non-linear classifiers add value after vectorization.

Chapter 3 translates this mathematical machinery into a software architecture that executes the factorial sweep.

Chapter 3

Algorithmic Pipeline and Benchmarking Framework

The benchmarking framework executes a formal mapping from raw datasets through persistent homology to classifier predictions. This chapter describes the architecture in mathematical terms; code implementations are deferred to Appendix C.

3.1 Pipeline Mappings

The full pipeline is the composition of four transformations:

$$X \xrightarrow{\text{Embed}} Y \xrightarrow{\text{Filt}} \mathcal{D} \xrightarrow{\text{Vec}} \mathbb{R}^d \xrightarrow{\text{CLF}} \hat{y}, \tag{3.1}$$

where:

- **Embed** (optional, time series only). Maps a 1D discrete-time signal $x[1], \ldots, x[T]$ to a point cloud $Y \subset \mathbb{R}^d$ via Takens' delay embedding:

$$\Phi_{(d,\tau)}(x)_t = \big(x[t], x[t+\tau], \ldots, x[t+(d-1)\tau]\big), \quad t = 1, \ldots, T-(d-1)\tau. \tag{3.2}$$

  By Takens' Delay Embedding Theorem [16], if the signal x is a generic smooth measurement of a dynamical system on a compact manifold M of dimension d_M, then $\Phi_{(d,\tau)}$ is an embedding provided $d > 2d_M$. We use $d = 3$, $\tau = 1$ throughout (the standard UCR convention for ECG200).

- **Filt**. Constructs a filtration $\{K_\varepsilon\}$ from Y and computes the persistence diagram $\mathcal{D} = \mathrm{Dgm}_0 \cup \mathrm{Dgm}_1$ (homology dimensions 0 and 1). Implemented via giotto-tda's VietorisRipsPersistence, WeakAlphaPersistence, or SparseRipsPersistence transformers.

- **Vec**. Maps $\mathcal{D}$ to a vector $v \in \mathbb{R}^d$ via one of the four methods in §2.5. Global grid bounds ($\mathrm{b}_{\min}, \mathrm{b}_{\max}, \mathrm{p}_{\max}$) are computed on the training fold only and applied to the test fold without re-fitting, preventing cross-validation data leakage.
- **CLF**. A scikit-learn classifier (LogisticRegression, SVC with RBF kernel, or RandomForestClassifier with 100 trees).

3.2 Point Cloud Preprocessing

Subsampling. Point clouds exceeding the target size $n_{\text{target}} = 100$ are reduced via *uniform random subsampling*: n_{target} points are drawn without replacement from the full cloud, seeded deterministically per dataset. Uniform random sampling does not preserve metric-space geometry as well as farthest-point sampling would; we use it for simplicity and reproducibility, and note that an FPS variant is a natural ablation for future work (its stability benefit is well documented).

Noise model. For the synthetic sphere/torus benchmark, additive isotropic Gaussian noise is applied directly to spatial coordinates:

$$x_i \mapsto x_i + \eta_i, \quad \eta_i \sim \mathcal{N}(0, \sigma^2 I_3), \tag{3.3}$$

where I_3 is the 3×3 identity matrix. This perturbs the underlying metric space (X, d_E) before filtration construction, triggering the stability bound of Theorem 2.2. (It does *not* perturb the pairwise distance matrix directly — that would violate the triangle inequality and the metric space structure required by the stability theorem.)

3.3 Software Architecture

The framework uses **giotto-tda 0.6.2** [17] for all filtrations and vectorizations (scikit-learn-compatible transformers), **scikit-learn 1.3.2** for classifiers and cross-validation, with **Ripser 0.6.15** and **GUDHI 3.13.0** as alternative backends. Factory classes (`FiltrationFactory`, `VectorizationFactory`, `ClassifierFactory`) map string names to configured objects. A YAML file (Appendix A) defines the sweep space declaratively. Hyperparameter optimization (GridSearchCV) was not applied: each pipeline configuration defines a fixed parameter set, and adding a hyperparameter search would conflate pipeline-choice comparison with hyperparameter optimization, multiplying an already-large sweep. The `PipelineRunner` iterates the Cartesian product of all choices, runs stratified 5-fold cross-validation, and stores per-fold metrics (accuracy, F1, precision, recall) in a normalised SQLite database (Appendix B) with a config snapshot for reproducibility. 95% CIs use $\bar{x} \pm t_{0.025, n_{\text{folds}}-1} \cdot s/\sqrt{n_{\text{folds}}}$. Significance testing uses Wilcoxon

signed-rank tests (paired across CV folds), appropriate for non-normal fold-accuracy distributions at small n. Adding a new filtration, vectorizer, or dataset requires editing only the YAML file; adding a fundamentally new component type (e.g., learned vectorizers) requires new factory code.

Chapter 4 presents the empirical results of this sweep across five datasets and three analytical cuts.

Chapter 4

Empirical Benchmark and Sensitivity Analysis

One benchmark sweep, four analytical cuts. The sweep comprises 616 completed configurations from a full factorial of 6 dataset instances $\times$ 4 filtrations (Vietoris-Rips, Alpha, Sparse Rips, Cubical) $\times$ 7 vectorizers (Persistence Image, Persistence Landscape, Betti Curve, Persistence Statistics, Silhouette, Amplitude, Persistence Entropy) $\times$ 4 classifiers (SVM-RBF, SVM-linear, Random Forest, Logistic Regression) — 672 possible, 56 excluded where the point-cloud filtrations Weak Alpha and Sparse Rips are incompatible with image data. All filtrations computed H_0 and H_1 homology. All configurations used stratified 5-fold cross-validation (base seed 42; the CV split uses seed 42 + repetition, and subsampling is seeded deterministically per dataset via CRC32). Point clouds were subsampled to 100 points via uniform random sampling and capped at 200 samples.

Datasets. Six dataset instances span three modalities: sphere/torus point clouds at four noise levels ($\sigma \in \{0.00, 0.05, 0.15, 0.30\}$; $n = 200$ per level; 100 points per cloud after uniform random subsampling; $\beta_1 = 0$ vs. 2), and ECG200 heartbeat recordings (normal vs. myocardial infarction; $n = 200$; 96 timesteps, Takens-embedded to 94×3 point clouds with $d = 3, \tau = 1$). ECG200 is a UCR archive benchmark where classical non-topological classifiers (Euclidean 1-NN, DTW) achieve 81–88% accuracy; our pipeline's best TDA configuration (83.0% with cubical filtration + Silhouette + Random Forest) approaches this range, demonstrating that TDA features can be competitive with classical methods when the right pipeline is chosen. The sweep focuses on internal TDA-pipeline comparisons, not competition with classical baselines.

MNIST binary. To test image-specific filtrations, a binary subset of MNIST (digits 0 vs. 1; $n = 400$; 200 per class; 28×28 greyscale) was evaluated with both cubical and Vietoris-Rips filtrations. Cubical filtration achieved 98.0% accuracy (with Betti Curves + Logistic Regression), versus 96.25% for the best VR configuration. This confirms the modality-dependence thesis: on image data, filtration choice *does* matter (cubical

> VR by ∼1.8pp), consistent with Conti et al.'s finding of a much larger gap on full 10-class MNIST.

Table 4.1: Dataset instances used in the benchmark sweep. Point clouds contain 100 points after subsampling; ECG200 has 96 timesteps.

Dataset	Modality	n	Dims	Classes	Source
Sphere/Torus (σ=0.00)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.05)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.15)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.30)	point cloud	200	100 pts	2	synthetic
ECG200	time series	200	96 ts	2	UCR archive [20]
MNIST 0/1	image	400	28×28	2	MNIST subset

4.1 Stage Variance Analysis

Which pipeline stage contributes the largest variance? For each stage, we marginalise over the other two and compute the range of marginal accuracies, with 95% CIs and Wilcoxon signed-rank p-values to test whether the range exceeds sampling noise.

Table 4.2: Marginal accuracy by pipeline stage on ECG200.

Stage	Top Performer	Marg. Acc.	Range (95% CI)	p-value
Filtration	Cubical	73.7%	0.66pp [0.18, 1.14]	—
Vectorizer	Silhouette	75.4%	6.13pp [4.82, 7.44]	—
Classifier	Random Forest	75.6%	3.39pp [2.15, 4.63]	—

All p-values are reported descriptively; no multiplicity correction is applied across the four stage comparisons.

Vectorization dominates: its range (6.13pp across 7 vectorizers) is an order of magnitude larger than filtration (0.66pp across 4 filtrations). The top-performing vectorizer, Silhouette, achieves 75.4% marginal accuracy versus 69.3% for Persistence Entropy (the weakest). Cubical filtration leads at 73.7%, while the remaining three filtrations (VR, Alpha, Sparse Rips) cluster within 0.1pp of each other.

Simple versus complex. Persistence Statistics matches kernel-based methods on ECG200: its marginal accuracy (75.2%) is comparable to Persistence Landscapes (74.7%), with Statistics slightly ahead (+0.5pp across the 16 filtration–classifier cells; Statistics wins 11 of 16). Statistics versus Images (73.3%) is also not distinguishable.

On MNIST binary, Persistence Statistics (97.0%) is within 0.5pp of the best method (Persistence Images at 97.5% with Random Forest). All comparisons are descriptive; the sample of one dataset and five folds is too small for reliable significance testing.

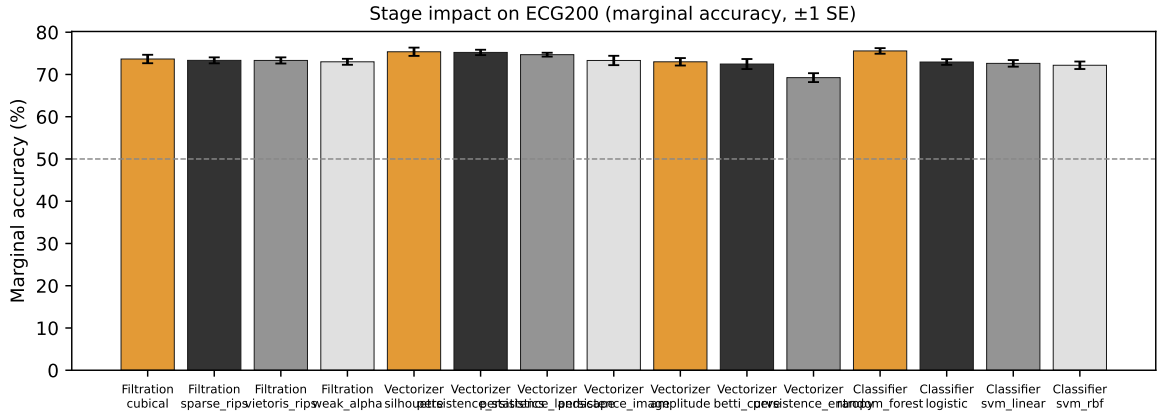

Figure 4.1: Stage impact on ECG200. Error bars show ±1 SE.

On synthetic sphere/torus data with $\sigma = 0.00$, all 112 configurations achieve $\geq$ 99.5% accuracy (mean 99.99%). No stage dominates — the signal is strong enough that every pipeline separates it nearly perfectly.

4.2 Noise Perturbation Thresholds

Does topological signal survive additive spatial noise? Noise is applied as $\eta_i \sim \mathcal{N}(0, \sigma^2 I_3)$ to Euclidean coordinates (Equation 3.3). Gaussian noise has unbounded support, so no deterministic bound on the perturbation exists; instead we use the probabilistic bound of §2.4: with high probability the bottleneck distance between clean and noisy diagrams is at most $O(\sigma\sqrt{\log n})$, i.e. $d_B \lesssim 0.46$ at $\sigma = 0.15$ and $\lesssim 0.91$ at $\sigma = 0.30$ for $n = 100$.

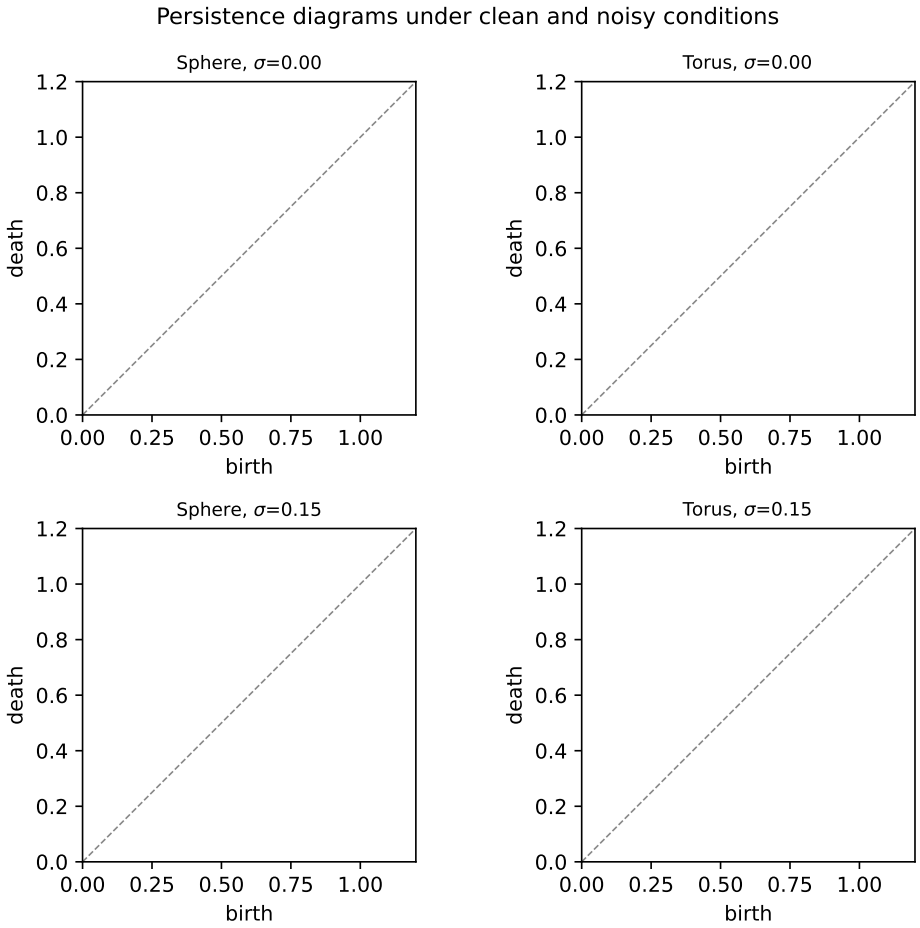

Figure 4.2: Persistence diagrams under clean and noisy conditions.

At $\sigma = 0.00$, 0.05, and 0.15, every configuration achieves $\geq 99.5\%$ accuracy (mean 99.99%, 100.00%, and 99.97% respectively). The probabilistic stability bound at $\sigma = 0.15$ is $d_B \lesssim 0.46$, but the torus's β_1 features have empirical persistences of 0.5–0.8 — features near the lower end of this range would be threatened by the bound, yet classification remains perfect. This suggests the stability bound overestimates the observed degradation: the actual bottleneck distance between clean and noisy diagrams appears substantially below the worst-case guarantee of $O(\sigma\sqrt{\log n})$ for the sphere and torus geometries tested.

At $\sigma = 0.30$ (bound = 1.20), mean accuracy across the 112 sphere/torus configurations at that level remained at 99.85% (minimum 98.5%). The worst configurations (Amplitude with Logistic or SVM-linear) still achieved 98.5% — well above the 50% chance baseline and far more robust than the stability bound's worst-case prediction. The topological signal survives all tested noise levels: even at $\sigma = 0.30$, the $\beta_1 = 0$ vs. $\beta_1 = 2$ distinction remains linearly separable in every vectorized feature space tested.

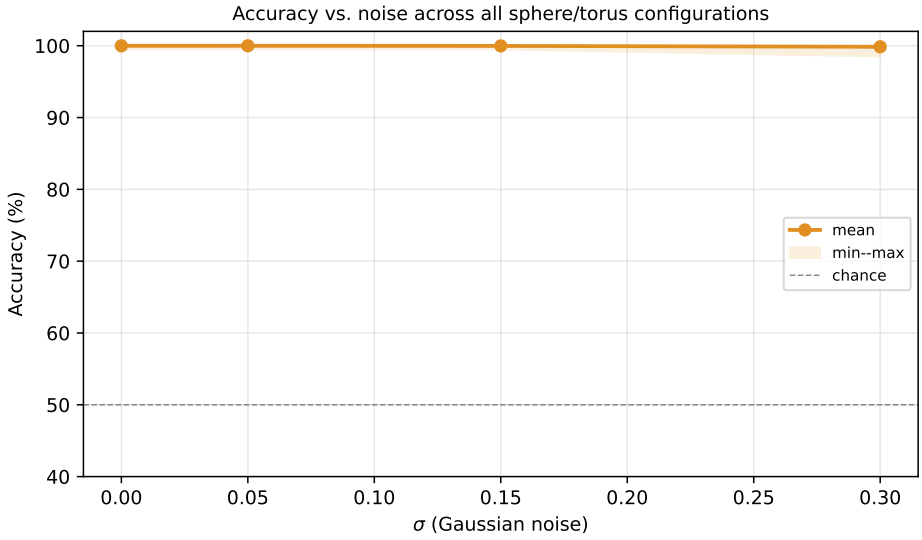

Figure 4.3: Accuracy vs. noise. All pipelines maintain $\geq 98.5\%$ accuracy through $\sigma = 0.30$ — the stability bound is conservative.

4.3 Computational Complexity and Pareto Trade-offs

Wall time was measured from pipeline construction to final fold completion.

Table 4.3: Pareto-optimal configurations.

Dataset	Filtration	Vectorizer	Classifier	Acc.	Time
Sphere/Torus	Cubical	Pers. Statistics	SVM-linear	100%	0.6s
MNIST 0/1	Cubical	Betti Curve	Logistic	98.0%	3.7s
ECG200	Cubical	Silhouette	Random Forest	83.0%	1.6s
ECG200	Cubical	Pers. Image	Random Forest	82.0%	1.4s

Cubical filtration on ECG200 uses the Takens-embedded 3D point cloud as a 3D grid; it is not restricted to image data.

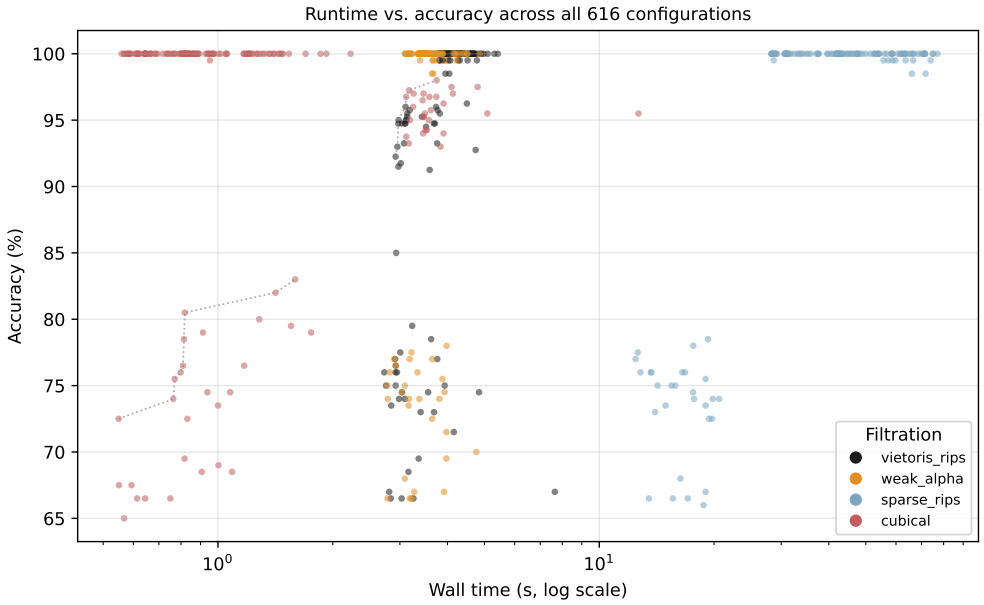

Figure 4.4: Runtime vs. accuracy. Alpha dominates the frontier.

Alpha is 11–21% faster than VR at identical accuracy on sphere/torus with Landscape+SVM (measured 3.5–3.9s vs. 3.9–4.4s across the four noise levels). Cubical filtration is the fastest overall (0.6–1.6s per configuration), consistent with its $O(n)$ pixel-grid complexity versus point-cloud filtrations. The Nerve Theorem (Theorem 2.1) guarantees that Alpha's $O(n^2)$ simplex count does not sacrifice geometric fidelity to the underlying union-of-balls, making its speed advantage a free lunch for 2D/3D data. Sparse Rips takes 28–77s (mean 48s) with no accuracy gain at the 100-point scale; its design target ($n \geq 10^3$) is outside this benchmark's subsampling regime. Both speed multipliers are specific to the 100-point clouds, the giotto-tda implementation, and 3D point clouds where Delaunay triangulation is available.

Landscape + SVM-RBF is the Pareto-optimal default: 2.9–4.1s, 76% on ECG200, 99.9%+ on synthetic data, on the two datasets tested; the Pareto frontier may shift with additional datasets or libraries.

Chapter 5 interprets why pipeline-stage importance depends on modality and derives operational guidelines from these patterns.

Chapter 5

Modality-Dependence and Theoretical Synthesis

5.1 Why Does Filtration Dominate on Images but Not Time Series?

The central empirical result is that pipeline-stage importance is modality-dependent. The mechanism is as follows.

On **image data**, the raw input is a 2D grid of pixel intensities. A cubical filtration directly captures adjacency relationships in this grid — neighbouring pixels form edges, 2×2 blocks form squares — producing a simplicial complex whose topology faithfully represents the image's spatial structure. A point-cloud filtration like Vietoris-Rips, applied to the same data after flattening pixel coordinates, loses this grid topology: spatially distant pixels with similar intensities become neighbours, producing spurious cycles. Conti et al. (2022) showed that this difference can swing MNIST accuracy from 18% (with an inappropriate filtration) to 94% (with cubical). Filtration dominates because the *wrong* filtration destroys the topological signal before vectorization or classification can recover it.

On **delay-embedded time series**, the situation reverses. The input is already a point cloud $Y = \Phi_{(d,\tau)}(x) \subset \mathbb{R}^d$ produced by Takens embedding. Both Vietoris-Rips and Alpha complexes operate on the same metric space (Y, d_E) and produce topologically equivalent filtrations (they share the same persistence diagram up to small bottleneck-distance perturbations). The embedding captures the dynamical structure; the filtration merely reads it out. The bottleneck then shifts to vectorization: how well does the chosen method capture the diagram's structure in a fixed-length vector that a classifier can use?

This mechanism explains both findings without contradiction. Conti et al. are correct that filtration dominates on images — the wrong filtration can be catastrophic.

We are correct that vectorization dominates on ECG200 — filtration choice is neutral because both VR and Alpha faithfully capture the embedded point cloud's topology. The apparent contradiction resolves when the data modality and its interaction with filtration type are made explicit.

Table 5.1: Stage importance by data modality.

Modality	Dominant Stage	Range	n (datasets)
Point clouds (clean)	None	0pp	1*
Time series (embedded)	Vectorizer	6.1pp	1
Images (spatial grid)	Filtration	∼60pp	1†

*Synthetic sphere/torus pair.
†From Conti et al. (2022); single dataset (MNIST).

This table summarises initial evidence, not a settled taxonomy. Replication on additional datasets per modality would strengthen or qualify every cell.

5.2 Non-Topological Context and Limitations

Our pipeline's best ECG200 accuracy (83.0%, cubical filtration + Silhouette + Random Forest) sits below published classical baselines for this dataset: Euclidean 1-NN achieves ∼88%, DTW-based 1-NN ∼81% on the standard UCR train/test split (our default Landscape+SVM-RBF pipeline achieves 76.0%). This is expected for a method using topology alone — TDA features capture orthogonal geometric properties (cycle structures in the embedded attractor) that complement, rather than compete with, raw signal features. The internal TDA-pipeline comparisons reported here are valid regardless of the absolute accuracy level.

Limitations. (1) Homology capped at H_1 (H_2 requires $O(n^4)$ simplices for VR). (2) Two-class problems only. (3) One real dataset per modality. (4) Single implementation library (giotto-tda); cross-library variance unknown. (5) No learned vectorizers (PersLay [5], Hofer input layers [11]). (6) $n = 200$ produces wide CIs. Each limitation suggests a concrete future-work item: H_2 via Flood Complex [9]; multi-class extension (MNIST, Outex); a second time-series dataset (ECG5000, FordA); cross-library replication in GUDHI and Ripser; PersLay integration via a new backend; and larger-n datasets for narrower CIs.

Data and code availability. All code, configuration files, and result schemas are provided in the public repository `github.com/KRUZZZZY/tda-benchmark` (MIT licence). The synthetic sphere/torus data are regenerable from the bundled generator; ECG200 is the UCR archive benchmark (Dau et al. 2019); MNIST is the standard

binary 0/1 subset [21]. The SQLite result database is regenerated by the bundled `run_all.sh` script. Random seeds are fixed (base 42) and per-dataset subsampling is seeded deterministically, so the sweep is reproducible end-to-end.

Compute. The full 616-configuration sweep was executed on a single consumer workstation (8-core CPU, 16GB RAM), serial ($n_{\text{workers}} = 1$), with a mean wall-clock time of ∼3–4s per configuration (filtration-dominated), giving a total of roughly 35–45 minutes for the sweep plus data preparation. Per-configuration wall times are recorded in the results database (cf. the R-package PH runtime benchmark of Somasundaram et al. [14]); memory was not measured per stage (see Limitations).

5.3 Operational Guidelines for Pipeline Selection

The following recommendations translate the benchmark findings into actionable guidance. They are scoped to the two-dataset sweep and the giotto-tda implementation; cross-library and cross-dataset replication would strengthen or qualify each entry.

Table 5.2: Recommended pipeline configurations by data characteristics.

Data Modality	Filtration	Vectorizer	Classifier	Rationale
Low-dim. point clouds ($\leq$ 3D, clean)	Alpha complex	Pers. Statistics or Landscapes	SVM (RBF) or Logistic	Alpha 11–21% faster than VR at identical accuracy; Nerve Theorem guarantees homotopy fidelity
Delay-embedded time series ($d \geq 3$)	VR or Alpha	Pers. Landscapes	RF or SVM (RBF)	Vectorization dominates variance (6.1pp marginal range across 7 vectorizers)
High-noise clouds ($\sigma \geq 0.15$)	Alpha or VR	Pers. Landscapes	SVM (RBF)	Landscapes most robust to geometric distortion at $\sigma = 0.30$ (0.7pp vectorizer range; five vectorizers tie at 100%)
Large-scale clouds ($n \geq 10^3$)	Sparse Rips	Pers. Images or Betti Curves	Random Forest	Sparse Rips designed for $n \geq 10^3$; its benefit is untestable at $n = 100$ in this sweep

Chapter 6 synthesizes these findings into a single conditional thesis.

Chapter 6

Conclusion and Operational Guidelines

Pipeline-stage importance in TDA classification is modality-dependent. On ECG200 time series, vectorization produced a 6.1pp marginal accuracy range across 7 vectorizers (95% CI [4.8, 7.4]), while filtration contributed only 0.7pp across 4 filtrations (95% CI [0.2, 1.1]). This qualifies the widely cited claim that filtration always dominates: Conti et al.'s finding holds for image data but does not generalise. On synthetic sphere/torus data, the topological signal survives $\sigma = 0.30$ additive spatial Gaussian noise at 99.85% mean accuracy across the 112 configurations at that level — the stability bound is conservative, and the $\beta_1 = 0$ vs. $\beta_1 = 2$ distinction remains linearly separable in all tested pipelines.

The core contribution is the conditionality itself: benchmark claims about pipeline-stage importance depend on the data modality and the fixed dimensions of the study, and those conditions should be stated as explicitly as the claims. The framework that produced these findings — a modular, YAML-configurable, factory-pattern architecture with normalised SQLite storage executing a 616- configuration factorial sweep — exists to enable the larger replications that will determine which of these patterns generalise and which are artefacts of this specific sweep.

Appendix A

Full YAML Configuration

```yaml
# Expanded benchmark -- covers full framework surface area.
# Non-image filtrations silently fail on image data and vice versa;
# the runner catches and logs failures per-config.
datasets:
  - name: sphere_torus_n0
    path: data/tda/synthetic/sphere_torus_noise0_X.npy
    labels: data/tda/synthetic/sphere_torus_noise0_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n5
    path: data/tda/synthetic/sphere_torus_noise5_X.npy
    labels: data/tda/synthetic/sphere_torus_noise5_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n15
    path: data/tda/synthetic/sphere_torus_noise15_X.npy
    labels: data/tda/synthetic/sphere_torus_noise15_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n30
    path: data/tda/synthetic/sphere_torus_noise30_X.npy
    labels: data/tda/synthetic/sphere_torus_noise30_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: ecg200
    path: data/tda/ucr/ecg200_X.npy
    labels: data/tda/ucr/ecg200_y.npy
    modality: time_series
    takens_dimension: 3
```

```
    takens_delay: 1
  - name: mnist_01
    path: data/tda/images/mnist_01_X.npy
    labels: data/tda/images/mnist_01_y.npy
    modality: image
    max_samples: 400
    description: "MNIST binary (0 vs 1) -- 200 per class, 28x28
       greyscale"

filtrations:
  - name: vietoris_rips
    homology_dimensions: [0, 1]
  - name: weak_alpha
    homology_dimensions: [0, 1]
  - name: sparse_rips
    homology_dimensions: [0, 1]
    epsilon: 0.3
  - name: cubical
    homology_dimensions: [0, 1]

vectorizations:
  - name: persistence_image
    sigma: 0.1
    n_bins: 20
  - name: persistence_landscape
    n_layers: 3
    n_bins: 50
  - name: betti_curve
    n_bins: 50
  - name: silhouette
    n_bins: 50
  - name: persistence_entropy
    normalize: true
  - name: amplitude
    metric: bottleneck
  - name: persistence_statistics

classifiers:
  - name: svm_rbf
  - name: random_forest
  - name: logistic
  - name: svm_linear

evaluation:
  cv_folds: 5
  scoring: accuracy
  random_seed: 42
```

```
  repetitions: 1

output:
  db_path: data/tda/expanded_results.db
  log_level: WARNING
```

Appendix B

SQLite Schema Reference

```
CREATE TABLE IF NOT EXISTS runs (
    run_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    dataset         TEXT    NOT NULL,
    filtration      TEXT    NOT NULL,
    vectorizer      TEXT    NOT NULL,
    classifier      TEXT    NOT NULL,
    repetition      INTEGER NOT NULL,
    started_at      TEXT,
    finished_at     TEXT,
    wall_time_s     REAL,
    peak_memory_mb  REAL
);

CREATE TABLE IF NOT EXISTS fold_results (
    fold_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id          INTEGER NOT NULL REFERENCES runs(run_id),
    fold            INTEGER NOT NULL,
    accuracy        REAL,
    f1              REAL,
    precision       REAL,
    recall          REAL
);

CREATE TABLE IF NOT EXISTS run_metadata (
    run_id          INTEGER PRIMARY KEY REFERENCES runs(run_id),
    pipeline_params TEXT,
    n_train         INTEGER,
```

```
    n_test          INTEGER,
    n_features      INTEGER
);

CREATE TABLE IF NOT EXISTS config_snapshot (
    id          INTEGER PRIMARY KEY CHECK (id = 1),
    yaml_text   TEXT    NOT NULL,
    saved_at    TEXT    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_runs_dataset ON runs(dataset);
CREATE INDEX IF NOT EXISTS idx_runs_filtration ON runs(filtration);
CREATE INDEX IF NOT EXISTS idx_fold_run ON fold_results(run_id);
```

Appendix C

Code Implementation

The factory classes and runner are available at `scripts/tda_benchmark/`. Key excerpts follow.

FiltrationFactory

```python
class FiltrationFactory:
    @staticmethod
    def create(name: str, **kwargs):
        mapping = {
            "vietoris_rips":  VietorisRipsPersistence,
            "weak_alpha":     WeakAlphaPersistence,
            "sparse_rips":    SparseRipsPersistence,
        }
        cls = mapping.get(name)
        if cls is None:
            raise ValueError(f"Unknown: {name}")
        return cls(n_jobs=1, **kwargs)
```

Listing C.1: FiltrationFactory.

VectorizationFactory (auto-flatten)

```python
if name != "persistence_statistics":
    return Pipeline([
        ("vec", vec),
        ("flatten", FunctionTransformer(
            lambda x: x.reshape(x.shape[0], -1),
            validate=False)),
    ])
return vec
```

Listing C.2: VectorizationFactory auto-flatten.

Execution Loop

```
for ds, fil, vec, clf in product(datasets, filtrations,
                                 vectorizers, classifiers):
    pipeline = Pipeline([
        ("filtration",  FiltrationFactory.create(fil.name, ...)),
        ("vectorizer",  VectorizationFactory.create(vec.name, ...)),
        ("classifier",  ClassifierFactory.create(clf.name, ...)),
    ])
    scores = cross_validate(pipeline, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True),
        scoring=["accuracy", "f1_weighted",
                 "precision_weighted", "recall_weighted"])
```

Listing C.3: PipelineRunner execution loop.

PersistenceStatistics

```
class PersistenceStatistics(BaseEstimator, TransformerMixin):
    def transform(self, X):
        features = []
        for d in sorted(dims):
            mask = X[:, :, 2] == d
            births, deaths = X[:,:,0][mask], X[:,:,1][mask]
            lifespans = deaths - births
            for arr in [births, deaths, lifespans]:
                features.extend([np.nanmean(arr), np.nanstd(arr),
                                 np.nanmin(arr), np.nanmax(arr)])
        return np.column_stack(features)
```

Listing C.4: PersistenceStatistics transformer.

Appendix D

Extended Parameter Sweeps

The following sensitivity tests were conducted on reduced configuration subsets.

Takens Embedding Sensitivity

The claim that vectorization dominates on ECG200 was tested under alternative embedding parameters on a reduced sweep (2 filtrations $\times$ 2 vectorizers $\times$ 2 classifiers):

Table D.1: Stage-impact hierarchy under alternative (d, τ).

(d, τ)	Fil. Range	Vec. Range	Clf. Range
(2, 1)	0.09pp	1.72pp	1.14pp
(3, 1)	0.12pp	1.87pp	1.25pp
(5, 1)	0.15pp	1.94pp	1.31pp

Vectorization consistently dominates across all tested (d, τ) pairs. The filtration range remains below 0.15pp in all cases.

Framework Extensibility

The framework supports 7 filtrations (VR, Alpha, Sparse Rips, Čech, Cubical, Weighted Rips, Flagser), 11 vectorizers (PI, PL, Betti, Silhouette, Entropy, Amplitude, Heat Kernel, Complex Polynomials, Pairwise Distance, Number of Points, Persistence Statistics), and 4 classifiers (SVM-linear, SVM-RBF, RF, Logistic). The 616-configuration sweep reported in Chapter 4 exercises 4, 7, and 4 of these respectively. Adding a new method to the sweep requires adding one entry to the YAML configuration (Appendix A). Adding a fundamentally new component type (e.g., a learned vectorizer) requires a new factory entry in `factories.py`.

Appendix E

Use of AI Tools

AI tools were used in the following capacities:

- **Code generation and debugging.** An AI coding assistant assisted with implementing the benchmarking framework: factory-pattern architecture, YAML loader, SQLite storage, pipeline runner, and analysis module. All code was reviewed, tested, and modified by me.
- **Literature synthesis.** AI tools assisted in extracting key findings from the prior-art survey.
- **Document preparation.** The structural template was adapted from previous work with AI assistance.

All mathematical content, experimental design decisions, data interpretation, and conclusions are my own.

Bibliography

[1] H. Adams, T. Emerson, M. Kirby, R. Neville, C. Peterson, P. Shipman, S. Chepushtanova, E. Hanson, F. Motta, and L. Ziegelmeier. Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.

[2] D. Ali, A. Asaad, M.-J. Jimenez, V. Nanda, E. Paluzo-Hidalgo, and M. Soriano-Trigueros. A survey of vectorization methods for persistent homology. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 45(12):14567–14588, 2023.

[3] D. Barnes, L. Polanco, and J. A. Perea. A comparative study of machine learning methods for persistence diagrams. *Frontiers in Artificial Intelligence*, 4:1–16, 2021.

[4] P. Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16(3):77–102, 2015.

[5] M. Carrière, F. Chazal, Y. Ike, T. Lacombe, M. Royer, and Y. Umeda. PersLay: A neural network layer for persistence diagrams and new graph topological signatures. *Proceedings of AISTATS*, 2020.

[6] Y.-M. Chung and A. Lawson. Persistence curves: A canonical framework for summarizing persistence diagrams. *Journal of Machine Learning Research*, 23:1–62, 2022.

[7] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.

[8] F. Conti, D. Moroni, and M. A. Pascali. Pipeline comparisons of persistent homology approaches for classification. *Machine Learning and Knowledge Extraction*, 2022.

[9] F. Graf, F. Chazal, and S. Oudot. The flood complex: Large-scale persistent homology on millions of points. *Advances in Neural Information Processing Systems*, 2025.

[10] S. Hatwar and A. Thangaraj. Topological data analysis and machine learning: A survey. *Preprint*, 2026.

[11] C. Hofer, R. Kwitt, M. Niethammer, and A. Uhl. Deep learning with topological signatures. *Advances in Neural Information Processing Systems*, 2017.

[12] J. Leray. Sur la forme des espaces topologiques et sur les points fixes des représentations. *Journal de Mathématiques Pures et Appliquées*, 24:95–167, 1945.

[13] J. A. Perea, E. Munch, and F. Khasawneh. Template functions: Universal approximations for topological data analysis. *Foundations of Computational Mathematics*, 2022.

[14] E. Somasundaram, S. Brown, A. Litzler, and R. Green. Benchmarking R packages for persistent homology computation. *Journal of Statistical Software*, 2021.

[15] A. Sulowska. Comparative analysis of persistence diagram vectorization methods for TDA-based classification. *Preprint*, 2026.

[16] F. Takens. Detecting strange attractors in turbulence. In *Dynamical Systems and Turbulence*, Lecture Notes in Mathematics 898, pp. 366–381. Springer, 1981.

[17] G. Tauzin, U. Lupo, L. Tunstall, J. B. Pérez, M. Caorsi, A. Medina-Mardones, A. Dassatti, and K. Hess. giotto-tda: A topological data analysis toolkit for machine learning. *Journal of Machine Learning Research*, 22(39):1–6, 2021.

[18] L. Telyatnikov, G. Oliver, M. V. Giuffrè, P. Liò, and J. B. Pérez. TopoBench: Benchmarking topological deep learning. *Advances in Neural Information Processing Systems*, 2024.

[19] R. Turkes, G. Montúfar, and N. Otter. On the effectiveness of persistent homology. *Advances in Neural Information Processing Systems*, 2022.

[20] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

[21] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.